Chuleta: Práctica Dockerfile (ejercicios resueltos)

Se usa el usuario **jose93ln** de Docker Hub. Si el tuyo es otro, cámbialo en todos los comandos.

0. Conceptos clave

Concepto	Qué es
Imagen	Plantilla de solo lectura con el sistema y el programa ya instalados.
Contenedor	Una imagen en ejecución. De una imagen puedes lanzar muchos contenedores.
Dockerfile	Archivo de texto con las instrucciones para construir una imagen.
Contexto de build	La carpeta donde ejecutas `docker build`. Docker solo puede copiar archivos de esa carpeta.
Tag	La versión de la imagen (lo que va detrás de los dos puntos).

Gestor de paquetes según la imagen base (lo dice el enunciado):

Si el tag lleva...	Gestor	Comando
alpine	apk	apk add --no-cache PAQUETE
nada (Debian)	apt-get	apt-get update && apt-get install -y PAQUETE

Carpeta donde va el index.html:

Servidor	Carpeta
NGINX	/usr/share/nginx/html/
Apache (httpd)	/usr/local/apache2/htdocs/

1. Preparación

```
mkdir -p ~/docker-imatges/{ex1,ex2,ex3,ex4}
cd ~/docker-imatges
cp ~/index.html ex1/
cp ~/index.html ex2/
cp ~/index.html ex3/
cp ~/index.html ex4/
```

- `mkdir` crea carpetas; `-p` crea también las carpetas padre y no da error si ya existen.
- `{ex1,ex2,ex3,ex4}` es una expansión de llaves: crea las cuatro carpetas de una vez.
- `cd` cambia de directorio. ~ es tu carpeta personal (home).
- `cp origen destino` copia un archivo. Cambia `~/index.html` por la ruta real del archivo del profesor.
- Cada carpeta necesita su copia porque el COPY del Dockerfile solo ve la carpeta donde estás.

2. Ejercicio 1: nginx:1.27-alpine + nano

```
cd ~/docker-imatges/ex1
nano Dockerfile
```

nano Dockerfile abre el editor y crea el archivo. Guardar: **Ctrl+O**, Enter. Salir: **Ctrl+X**. Debe llamarse exactamente `Dockerfile` (D mayúscula, sin extensión).

```
FROM nginx:1.27-alpine
RUN apk add --no-cache nano
COPY index.html /usr/share/nginx/html/index.html
```

Línea	Qué hace
`FROM nginx:1.27-alpine`	Imagen base exacta que pide el enunciado.
`RUN apk add --no-cache nano`	Ejecuta un comando **durante la construcción**. Instala nano con apk (porque el tag es alpine). `--no-cache` no guarda la caché de paquetes y la imagen pesa menos.
`COPY index.html /usr/share/nginx/html/index.html`	Copia tu página a la carpeta que sirve NGINX.

```
docker build -t jose93ln/web-nginx:1.0 .
docker run -d --name web1 -p 8081:80 jose93ln/web-nginx:1.0
```

- `docker build` construye una imagen a partir de un Dockerfile.
- `-t usuario/nombre:tag` le pone nombre a la imagen. El **punto final** es el contexto (carpeta actual): no lo olvides.
- `docker run` crea y arranca un contenedor. `-d` lo deja en segundo plano.
- `--name web1` nombre del contenedor (lo exige el enunciado).
- `-p 8081:80` = PUERTO_TU_MAQUINA:PUERTO_CONTENEDOR. NGINX escucha en el 80 dentro; tú accedes por el 8081.
- `curl` comprueba que sirve tu página. `docker exec` ejecuta un comando dentro de un contenedor en marcha; si nano responde con versión, está instalado.

```
curl http://localhost:8081
docker exec web1 nano --version
```

3. Ejercicio 2: httpd:2.4.62 + curl

```
cd ~/docker-imatges/ex2
nano Dockerfile
```

nano Dockerfile abre el editor y crea el archivo. Guardar: **Ctrl+O**, Enter. Salir: **Ctrl+X**. Debe llamarse exactamente `Dockerfile` (D mayúscula, sin extensión).

```
FROM httpd:2.4.62
RUN apt-get update && apt-get install -y curl
COPY index.html /usr/local/apache2/htdocs/index.html
```

Línea	Qué hace
`FROM httpd:2.4.62`	Imagen oficial de Apache. Sin alpine en el tag, es Debian.
`RUN apt-get update && apt-get install -y curl`	`apt-get update` descarga la lista de paquetes (sin esto no encuentra nada). && encadena: el segundo comando solo se ejecuta si el primero va bien. `-y` responde "sí" automáticamente.
`COPY ... /usr/local/apache2/htdocs/index.html`	Carpeta que sirve Apache.

```
docker build -t jose93ln/web-apache:1.0 .
docker run -d --name web2 -p 8082:80 jose93ln/web-apache:1.0
```

- Mismos comandos que antes con otro nombre, tag y puerto (8082). Apache también escucha en el puerto 80 interno.

```
curl http://localhost:8082
docker exec web2 curl --version
```

4. Ejercicio 3: nginx:1.26 + nano

```
cd ~/docker-imatges/ex3
nano Dockerfile
```

nano Dockerfile abre el editor y crea el archivo. Guardar: **Ctrl+O**, Enter. Salir: **Ctrl+X**. Debe llamarse exactamente `Dockerfile` (D mayúscula, sin extensión).

```
FROM nginx:1.26
RUN apt-get update && apt-get install -y nano
COPY index.html /usr/share/nginx/html/index.html
```

Línea	Qué hace
`FROM nginx:1.26`	Sin alpine: es Debian, así que se usa apt-get.
`RUN apt-get update && apt-get install -y nano`	Instala nano en Debian.
`COPY ...`	Carpeta de NGINX.

```
docker build -t jose93ln/web-nginx:2.0 .
docker run -d --name web3 -p 8083:80 jose93ln/web-nginx:2.0
```

- Mismo repositorio `web-nginx` que el ejercicio 1, pero con tag 2.0.

```
curl http://localhost:8083
docker exec web3 nano --version
```

5. Ejercicio 4: httpd:2.4-alpine + curl

```
cd ~/docker-imatges/ex4
nano Dockerfile
```

nano Dockerfile abre el editor y crea el archivo. Guardar: **Ctrl+O**, Enter. Salir: **Ctrl+X**. Debe llamarse exactamente `Dockerfile` (D mayúscula, sin extensión).

```
FROM httpd:2.4-alpine
RUN apk add --no-cache curl
COPY index.html /usr/local/apache2/htdocs/index.html
```

Línea	Qué hace
`FROM httpd:2.4-alpine`	Tag con alpine: se usa apk.
`RUN apk add --no-cache curl`	Instala curl en Alpine.
`COPY ...`	Carpeta de Apache.

```
docker build -t jose93ln/web-apache:2.0 .
docker run -d --name web4 -p 8084:80 jose93ln/web-apache:2.0
```

- Mismo repositorio `web-apache` que el ejercicio 2, pero con tag 2.0.

```
curl http://localhost:8084
docker exec web4 curl --version
```

6. Ejercicio 5: comparar tamaños

```
docker images
docker images | grep jose93ln
```

- `docker images` lista las imágenes con REPOSITORY, TAG, IMAGE ID, fecha y **SIZE**.
- `|` (pipe) pasa la salida al siguiente comando; `grep jose93ln` filtra solo tus cuatro imágenes.

Qué esperar (aproximado; mira tus datos reales):

Imagen	Base	Tamaño aprox.
web-nginx:1.0	nginx alpine	~50-60 MB
web-apache:2.0	httpd alpine	~60-70 MB
web-apache:1.0	httpd Debian	~150-170 MB
web-nginx:2.0	nginx Debian	~190-200 MB

Respuesta (ajústala a lo que veas en tu docker images):

La imagen más pequeña es jose93ln/web-nginx:1.0. Esto se debe a que está construida sobre Alpine Linux, una distribución mínima cuya imagen base pesa solo unos pocos MB. Las imágenes basadas en Debian incluyen muchas más librerías y utilidades del sistema, por lo que ocupan bastante más. Además, NGINX es más ligero que Apache y el paquete se instaló con apk add --no-cache, que no guarda caché.

7. Ejercicio 6: subir a Docker Hub

```
docker login -u jose93ln
docker push jose93ln/web-nginx:1.0
docker push jose93ln/web-apache:1.0
docker push jose93ln/web-nginx:2.0
docker push jose93ln/web-apache:2.0
```

- `docker login -u` inicia sesión; pide contraseña o un access token con permisos **Read & Write**.
- `docker push` sube una imagen. El nombre **debe empezar por tu usuario de Docker Hub**, si no da *insufficient_scope*.
- Se crean automáticamente los repositorios web-nginx y web-apache, cada uno con dos tags (1.0 y 2.0).
- **Comprobar:** hub.docker.com, tu cuenta, Repositories: deben aparecer los dos repos con sus dos tags.

8. Ejercicio 7: entrega en GitHub

README.md

```
cd ~/docker-imatges
nano README.md
```

```
# Práctica Dockerfile 1

**Alumno:** Tu Nombre Apellido

## Ejercicio 5: comparación de tamaños

La imagen más pequeña es `jose93ln/web-nginx:1.0`. Esto se debe a que
está construida sobre Alpine Linux, una distribución mínima cuya
imagen base pesa solo unos pocos MB. Las imágenes basadas en Debian
incluyen muchas más librerías y utilidades del sistema, por lo que
ocupan bastante más. Además, NGINX es más ligero que Apache y el
paquete se instaló con `apk add --no-cache`, que no guarda caché.
```

Crear el repo en GitHub (en la web)

New repository, nombre **practica-dockerfile-1**, **vacío** (sin README ni .gitignore, para evitar conflictos).

Subir

```
cd ~/docker-imatges
git init
git add .
git commit -m "Práctica Dockerfile 1"
git branch -M main
git remote add origin https://github.com/TU_USUARIO_GITHUB/practica-dockerfile-1.git
git push -u origin main
```

- `git init` convierte la carpeta en un repositorio git (crea la carpeta oculta .git).
- `git add .` prepara todos los archivos para el commit.
- `git commit -m` guarda una "foto" del estado actual con un mensaje.
- `git branch -M main` renombra la rama a main (la que usa GitHub por defecto).
- `git remote add origin URL` enlaza tu carpeta con el repo de GitHub. Cambia TU_USUARIO_GITHUB por tu usuario.
- `git push -u origin main` sube la rama; `-u` recuerda el destino para futuros push.

- Si pide contraseña: usa un **Personal Access Token** (Settings, Developer settings, Personal access tokens).
- Si dice "Please tell me who you are": `git config --global user.name "Tu Nombre"` y `git config --global user.email "tu@email.com"`.

Comprobaciones antes de subir: `ls` debe mostrar ex1 ex2 ex3 ex4 README.md; `ls ex1` (y los demás) debe mostrar Dockerfile e index.html; `git status` muestra lo que se va a subir.

9. Comandos de ayuda y errores típicos

Comando	Para qué sirve
docker ps	Contenedores en ejecución.
docker ps -a	Todos los contenedores (también los parados).
docker logs web1	Ver los logs de un contenedor (útil si no arranca).
docker stop web1	Parar un contenedor.
docker rm -f web1	Borrar un contenedor (-f lo fuerza aunque esté en marcha).
docker rmi IMAGEN	Borrar una imagen.
docker images	Listar imágenes.

Error	Causa y solución
name "/web1" is already in use	Ya existe un contenedor con ese nombre: `docker rm -f web1` y repite.
port is already allocated	El puerto está ocupado: para el contenedor que lo usa o revisa que no repites puertos.
Dockerfile: no such file	No estás en la carpeta correcta o el archivo no se llama exactamente Dockerfile.
COPY failed: file not found	Falta el index.html en esa carpeta.
apt-get: not found / apk: not found	Gestor equivocado (apt en alpine o apk en Debian).
insufficient_scope al hacer push	El nombre de la imagen no empieza por tu usuario o el token no tiene permisos de escritura.

10. Resumen rápido (copiar y pegar)

```
# Ex1
cd ~/docker-imatges/ex1 && docker build -t jose93ln/web-nginx:1.0 . && docker run -d --name web1 -p 8081
# Ex2
cd ~/docker-imatges/ex2 && docker build -t jose93ln/web-apache:1.0 . && docker run -d --name web2 -p 808
# Ex3
cd ~/docker-imatges/ex3 && docker build -t jose93ln/web-nginx:2.0 . && docker run -d --name web3 -p 8083
# Ex4
cd ~/docker-imatges/ex4 && docker build -t jose93ln/web-apache:2.0 . && docker run -d --name web4 -p 808

# Comprobaciones
curl http://localhost:8081 ; docker exec web1 nano --version
curl http://localhost:8082 ; docker exec web2 curl --version
curl http://localhost:8083 ; docker exec web3 nano --version
curl http://localhost:8084 ; docker exec web4 curl --version

# Tamaños y publicación
docker images
docker login -u jose93ln
docker push jose93ln/web-nginx:1.0
docker push jose93ln/web-apache:1.0
docker push jose93ln/web-nginx:2.0
docker push jose93ln/web-apache:2.0
```